

WEEK 5

Benchmarking and Tuning

HPC & Cluster Computing

Course: Operating Systems 4 (HPC)
Module: EIOSY4A
Topic: HPC Benchmarking and Performance Tuning
Reading Time: ~15 Minutes

Learning Outcomes

By the end of this week, you should be able to:

1. **Analyze** key HPC performance constraints including FLOPs, memory bandwidth, and network latency.
2. **Apply** the Roofline Model to diagnose and classify computational codes as compute-bound or memory-bound.
3. **Execute and evaluate** the HPL benchmark, interpreting its practical results against theoretical Rpeak.
4. **Critique** memory bandwidth limitations by analyzing STREAM benchmark outputs.
5. **Formulate** basic code optimization strategies utilizing compiler flags and specialized numerical libraries.

Week Goal

Welcome to Week 5! Our cluster is built and managed. But is it actually *fast*? This week, we move from "it works" to "it performs." You will run the same benchmarks used to rank the Top500 supercomputers, giving you objective, industry-recognized measurements of your cluster's capabilities.

WEEK 4 WEEK 6

Assessment Alignment: The weekly quiz and practical assignments for this module

Overview

We have now built a cluster (Weeks 1-4) and written parallel programs for it (Weeks 5-6). But how do we know if our cluster is "fast"? How do we define "performance"? This week focuses on **Benchmarking**—the science of measuring system performance. We will explore the theoretical limits of hardware (the Roofline Model) and use industry-standard tools (HPL, STREAM) to quantify the capabilities of our infrastructure.

1. Performance Metrics

In HPC, "speed" is a vague term. We use specific metrics to quantify different aspects of system performance.

1.1 FLOPs (Floating Point Operations per Second)

This is the standard unit of **Compute Power**. It measures how many mathematical calculations (specifically, floating-point additions or multiplications) a processor can perform in one second.

Unit	Value	Magnitude	Typical Scale
GFLOPS	10^9	Billions	Standard laptop
TFLOPS	10^{12}	Trillions	Single GPU or HPC Node
PFLOPS	10^{15}	Quadrillions	Supercomputer
EFLOPS	10^{18}		Exascale tier

This measures **Data Movement**. It is the rate at which data can be transferred between memory and the CPU, or between nodes over the network. **Memory Bandwidth** determines how fast the CPU can read from RAM (e.g., 50 GB/s on DDR4), while **Network Bandwidth** dictates how fast nodes can communicate (e.g., 100 Gbps Infiniband).

1.3 Latency

This measures **Delay**. It is the time it takes for a single bit of data to travel from Source to Destination. In HPC, low latency is critical for synchronization in MPI programs. Measuring latency often reveals the overhead of the network stack.

Recall the **Hockney Model** ($T = \alpha + \beta n$) from Week 2. When benchmarking a network, we must distinguish between the startup cost (α) and the transfer cost (β). A system might have high bandwidth but poor latency, making it unsuitable for algorithms that require frequent, small-scale synchronization.

---

2. The Memory Wall and Roofline Model

A common misconception is that a faster CPU always makes code run faster. Real-world performance is often limited not by how fast the CPU calculates, but by how fast it can get data from memory. This is known as the **Von Neumann Bottleneck** or the **Memory Wall**.

2.1 Arithmetic Intensity

To understand performance limits, we calculate the **Arithmetic Intensity (AI)** of an algorithm:

$$AI = \frac{\text{Floating Point Operations (FLOPs)}}{\text{Bytes Accessed from Memory (Bytes)}}$$

- **Compute-Bound**: High AI. The code does extensive math on small data (e.g., Matrix Multiplication). Performance is limited by CPU Speed.
- **Memory-Bound**: Low AI. The code does simple math on massive data (e.g., Vector Addition). Performance is limited by RAM Speed.

2.2 The Roofline Model

The Roofline Model is a visual tool used to determine if a code is performing optimally. It plots Arithmetic Intensity (X-axis) against Performance (Y-axis). The **Horizontal Roof** represents the theoretical peak CPU speed, indicating the maximum possible calculation rate. The **Diagonal Roof** represents the peak Memory Bandwidth. Optimizing code involves moving the performance point **Up** (closer to the roof) or **Right** (increasing arithmetic intensity).

---

3. High Performance Linpack (HPL)

HPL is the most famous benchmark in the world. It is the sole metric used to rank the **TOP500** supercomputers.

3.1 What it Solves

HPL solves a dense linear system of equations of the form $Ax = b$ using LU decomposition with partial pivoting. It is an $O(N^3)$ algorithm operating on $O(N^2)$ data, meaning it has very high Arithmetic Intensity. As a result, HPL is typically **Compute-Bound** and achieves close to the theoretical peak performance of the CPU.

3.2 The HPL Input File (HPL.dat)

Tuning HPL requires adjusting parameters in `HPL.dat`. The combination of these values determines the problem size and how it is distributed across the cluster.

Parameter	Description	Tuning Notes
N (Problem Size)	The size of the matrix.	Larger N usually yields better performance (better CPU/Memory ratio).
NB (Block Size)	The size of data chunks sent to cache.	Critical for tuning; usually 128 or 256.
P x Q (Process Grid)	Arrangement of MPI ranks.	$P \times Q$ must equal total processes. usually $P \leq Q$.

4. STREAM Benchmark

While HPL measures the CPU, **STREAM** measures the Memory Subsystem. It performs simple vector operations to test sustained memory bandwidth.

4.1 The Four Kernels

These four operations characterize different memory access patterns.

Kernel	Operation	Purpose
Copy	`a[i] = b[i]`	Measures pure transfer rates.
Scale	`a[i] = q * b[i]`	Measures simple arithmetic.
Add	`a[i] = b[i] + c[i]`	Measures 2 reads + 1 store.
Triad	`a[i] = b[i] + q * c[i]`	Complex mix; best predictor for real apps.

For many real-world scientific applications (like Fluids or Weather simulation), the **STREAM Triad** result is a better predictor of actual performance than HPL.

5. HPCG (High Performance Conjugate Multi-Gradient)

HPL has been criticized for being "too easy" to optimize and not representative of modern complex physics. **HPCG** is a newer benchmark designed to stress the system differently. It uses sparse matrix operations and irregular memory access patterns, punishing systems with poor interconnects or high latency. It is now a standard companion to HPL.

---

6. Tuning Fundamentals

Before rewriting code, administrators use **Tuning** to get free performance gains.

6.1 Compiler Optimization Flags

The compiler (GCC, Intel ICC) translates C/Fortran into Machine Code.

Flag	Optimization Level	Use Case
`-O0`	None	Debugging (slower, but exact).
`-O2`	Moderate	Standard for production.
`-O3`	Aggressive	Maximum speed; vectorizes loops; may break strict math compliance.
`-march=native`	Architecture Specific	Uses instruction sets (AVX2, AVX-512) for the host CPU.

6.2 Libraries (BLAS/LAPACK)

Never write your own Matrix Multiplication function. Optimized libraries like **OpenBLAS** or **Intel MKL** are hand-tuned assembly code that is often 50-100x faster than generic C code.

Based on the Hockney model, we know that every message incurs a latency cost (α). If your benchmarking reveals that your code is spending more time in communication than calculation, consider **Message Packing** (or Buffer Aggregation). Instead of sending many small 1KB messages, buffer them into a single 1MB message. This pays the α cost once instead of 1000 times, significantly improving the effective bandwidth utilized by the application.

Section Checkpoint

7. Summary and Next Steps

This week, we moved from functionality to performance. We learned that a working cluster is not necessarily a fast one. We identified the critical metrics of **Throughput (FLOPS)** and **Bandwidth (GB/s)**, and used the **Roofline Model** to visualize whether our applications are Compute-Bound or Memory-Bound. We mastered standard benchmarks like **HPL** (The Top500 Standard) and **STREAM**, providing a baseline for all future optimization.

To conclude our performance study, we use the **Iso-efficiency Function** to evaluate the scalability of our cluster. It defines the relationship between the problem size (W) and the number of processors (P) required to maintain a constant efficiency (E):

$$W = f(P)$$

If an algorithm is perfectly scalable, the problem size only needs to grow linearly with the number of processors. This mathematical foundation allows us to predict the performance of our software before deploying it on expensive supercomputing hardware.

Preparing for Week 6

WEEK 4 WEEK 6

models (Shared Memory vs. Distributed Memory) and writing our first parallel code using **OpenMP**.

Checklist for Next Week

☐ Can you calculate the theoretical peak performance (Rpeak) of a CPU given its Clock Speed, Core Count, and FLOPs/Cycle?
☐ Why does **HPL** stress the CPU while **STREAM** stresses the Memory Bus?
☐ What does it mean to be "Memory Bound," and why does buying a faster CPU not solve this problem?
☐ How do compiler flags like `-O3` and `-march=native` "freely" improve performance?

Test Your Knowledge

Ready to check your understanding of this week's material? Take the interactive quiz now!

Start Quiz

Resources

- **Dongarra, J. et al.** (2024). *HPL - A Portable Implementation of the High-Performance Linpack Benchmark*. Available at: https://www.netlib.org/benchmark/hpl/ (Accessed: 22 December 2025).
- **McCalpin, J.D.** (1995). *Memory Bandwidth and Machine Balance in Current High Performance Computers*. IEEE TCAA Newsletter. Available at: https://www.cs.virginia.edu/stream/ (Accessed: 22 December 2025).

Vaal University of Technology - Operating Systems 4 (HPC) 2026